

MIPS Simulator: A Comprehensive Exploration of Design and Implementation

1. Introduction

This project embarks on a journey to demystify the intricacies of computer architecture by developing a functional MIPS simulator. While textbooks and lectures provide a theoretical foundation, simulators offer a hands-on, interactive, and risk-free environment to solidify understanding and foster experimentation.

The motivation behind this MIPS simulator stems from the desire to empower users with the ability to:

- **Write and Execute:** Translate abstract MIPS assembly code into tangible results, observing program behavior step-by-step.
- **Overview the Datapath:** explanation of the flow of data through the simulator's datapath, showing the connection between instruction and execution.
- **Analyze Program Behavior:** Gain insights into register modifications, memory interactions, and control flow changes, facilitating in-depth debugging and performance analysis.
- **Experiment with Architecture:** Explore architectural modifications or optimizations, experiencing firsthand the impact of design choices on program behavior and efficiency.

1.1 Project Scope: Defining the Boundaries of Exploration

This MIPS simulator, while designed for comprehensive learning, focuses on the core principles and instructions of the MIPS I architecture. The current iteration supports:

Instructions:

- **Arithmetic and Logic:** add, addi, sub, mul, slt, sll, ori
- **Memory Access:** lw, sw
- **Control Flow:** beq, j, jal, jr
- **System Calls:** syscall (supporting basic I/O and program termination)

Data Types:

- 32-bit signed integers

Memory Model:

- Linear memory with a fixed size (e.g., 1MB)
- No virtual memory or caching mechanisms (for clarity)

Assembler Capabilities:

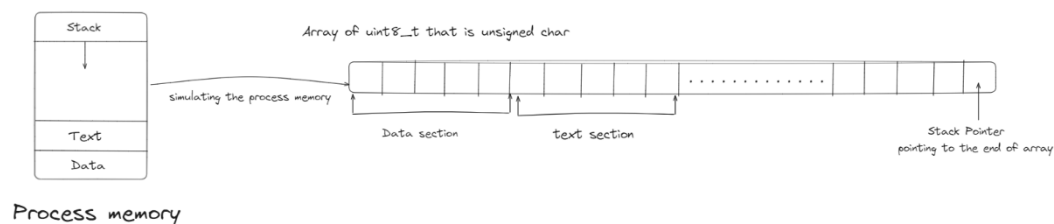
- Two-pass assembly with symbol table management
- Handling of basic directives: `.data`, `.ascii`, `.word`, `.space`
- Error detection and reporting
 - Invalid Instruction Mnemonics: Using an instruction name that is not part of the supported MIPS instruction set (e.g., `xyz $t0, $t1, $t2`).
 - Incorrect Number of Operands: Providing too few or too many operands for an instruction (e.g., `add $t0, $t1`).
 - Missing Commas or Parentheses: Errors in the use of delimiters (e.g., `lw $t0 4($t1)` – missing the comma).
 - Invalid Register Names: Using a register name that is not a valid MIPS register (e.g., `$tx`).

Limitations:

- No support for floating-point operations or coprocessor instructions.
- Pipelining, exception handling, and advanced memory management are not included in this version.

2. Architecture and Design

2.1 Detailed Datapath overview



- **Essential Components:** PC (Program Counter), Instruction Memory, Register File, ALU (Arithmetic Logic Unit), Data Memory, Multiplexers, Sign Extender, Adders (for PC manipulation).
- **Data Flow:** Depicts the movement of instructions and data through the datapath using arrows, highlighting each stage of the execution cycle.
- **Control Signals:** Indicates the origin of control signals from the Control Unit and label them clearly (e.g., `RegDst`, `ALUSrc`, `MemtoReg`), showing how they govern the datapath's behavior.

2.2 Control Unit Design

The main component of our simulator is the Control Unit, responsible for generating precise control signals that dictate the datapath's operation.

Instruction	regDst	Jump	Branch	MemRead	MemToReg	ALUOp	MemWrite	ALUSrc	RegWrite
add	10	00	0	0	01	000	0	0	1
addi	00	00	0	0	01	000	0	1	1
sub	10	00	0	0	01	001	0	0	1
mul	10	00	0	0	01	010	0	0	1
slt	10	00	0	0	01	101	0	0	1
ori	00	00	0	0	01	011	0	1	1
lui	00	00	0	0	01	100	0	1	1
lw	00	00	0	1	00	000	0	1	1
sw	XX	00	0	0	XX	000	1	1	0
beq	XX	00	1	0	XX	000	0	0	0
j	XX	01	X	0	XX	XXX	0	X	0
jal	01	01	X	0	10	XXX	0	X	1
jr	XX	10	X	0	XX	XXX	0	X	0

- **Inputs:** Opcode (6 bits), Function Code (6 bits) - extracted from the fetched instruction.
- **Outputs:** RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp (3 bits) - each output representing a specific control signal.

Explanation:

- **RegDst:** Selects the destination register for the Write Back stage (either the rt or rd field of the instruction).
- **ALUSrc:** Chooses the second operand for the ALU (either a register value or the sign-extended immediate from the instruction).
- **MemtoReg:** Determines the data to be written back to the register file (either ALU result or data read from memory).
- **RegWrite:** Enables or disables writing to the register file.
- **MemRead:** Controls read access to the data memory.
- **MemWrite:** Controls write access to the data memory.
- **Branch:** Indicates whether to update the PC based on a branch condition.
- **ALUOp:** Specifies the operation to be performed by the ALU (add, subtract, AND, OR, etc.).

- **Jump (PCsrc):** Specifies whether the next PC value comes from a jump instruction, a branch, or sequential execution.
- **MemtoReg:** Selects between ALU result or memory data to be written to a register.

2.3 Memory Organization: A Simplified View

We simulate a basic linear memory model, representing it as a contiguous array of bytes.

- **Size:** Fixed size (e.g., 1MB), sufficient for educational purposes and testing.
- **Address Space:** 32-bit addressing, allowing access to the entire memory range.
- **Storage:** Instructions and data share the same memory space.
- **Access:** Load (lw) and Store (sw) instructions interact directly with the memory array based on calculated addresses.

2.4 Register File Implementation: Storing the Essentials

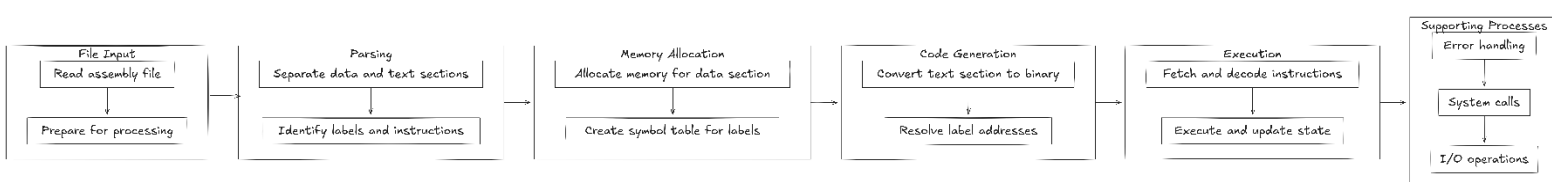
The register file, a critical component of the MIPS architecture, holds 32 general-purpose registers, each 32 bits wide.

- **Data Structure:** std::map provides an efficient key-value storage mechanism.
 - **Key:** 5-bit binary string representing the register number ("00000" to "11111").
 - **Value:** 32-bit integer representing the register content.
- **Read Access:** Two registers can be read simultaneously, driven by the Read Register 1 and Read Register 2 outputs from the instruction decoder.
- **Write Access:** Enabled by the RegWrite control signal. Data from the Write Data input is written to the register specified by the Write Register output.

3. Implementation Details

3.1 Assembler Algorithm: From Assembly to Machine Code

Mips Simulator Flow



Pass 1: Symbol Table Construction

1. Initialization:

- Create an empty symbol table (std::map).
- Set the initial instruction address to the starting address of the text segment (usually 0).

2. Line-by-Line Processing:

- Read each line of the assembly code.
- **Label Detection:** If a label is present:
 - Extract the label name.
 - Calculate its address based on the current instruction address.
 - Insert the label and its address into the symbol table.
- **Instruction Size Calculation:**
 - For each instruction or directive encountered, increment the instruction address by the appropriate size.

Pass 2: Machine Code Generation

1. Initialization:

- Reset the instruction address.
- Open the output file for writing (or prepare to directly write to the simulator's memory).

2. Line-by-Line Processing:

- Read each line of the assembly code.
- **Instruction Processing:**
 - **Opcode/Function Fetch:** Identify the opcode and function code based on the instruction mnemonic.
 - **Operand Extraction:** Extract register operands (rs, rt, rd) and immediate values (if any).
 - **Label Resolution:** For instructions referencing labels (e.g., beq, j), look up the label in the symbol table and retrieve its address.
 - **Binary Encoding:** Construct the complete 32-bit binary instruction, combining the opcode, function code, register operands (converted to their 5-bit binary representations), and either the sign-extended immediate value or the resolved label address.
- **Output:** Write the assembled binary instruction to the output file.

3.2 Handling of Directives: Guiding the Assembler

Directives provide instructions to the assembler, influencing memory organization and data allocation.

- **.data:** Signals the start of the data segment, updating the assembler's address pointer.
- **.text:** Marks the beginning of the text segment (where instructions reside).
- **.ascii:** Declares a null-terminated string. The assembler allocates memory and includes the null terminator.
- **.word:** Declares a 32-bit integer, allocating 4 bytes of memory.
- **.space:** Allocates a specified amount of memory, typically used for buffers or data structures.

3.3 Symbol Table Management: Efficient Label Lookup

The symbol table, implemented using `std::map`, plays a crucial role in resolving label addresses during the second pass of assembly.

- **Key:** Label name (string).
- **Value:** Memory address (integer).

The map's efficient search and insertion operations make it well-suited for this purpose.

3.4 Error Handling: Ensuring Code Integrity

Robust error handling is vital for any assembler. Our implementation incorporates checks for:

- **Syntax Errors:** Invalid instruction mnemonics, missing commas, incorrect register names.
- **Undefined Labels:** Referencing labels that do not exist in the symbol table.
- **Invalid Immediate Values:** Immediate values exceeding the allowable range for a specific instruction.
- **Duplicate Labels:** Two or more labels with the same name.
- **Address Overflow:** Data or instructions exceeding the allocated memory limits.

Upon encountering an error, the assembler displays a descriptive error message indicating the line number and error type, halting the assembly process to prevent the generation of incorrect code.

3.5 Simulator Execution Cycle: Emulating the Processor's Heartbeat

The simulator implements the fetch-decode-execute cycle, the fundamental process by which processors execute instructions.

1. Fetch:

- The instruction at the memory address pointed to by the PC (Program Counter) is retrieved.
- PC is incremented by 4 (assuming each instruction occupies 4 bytes).

2. Decode:

- The 32-bit instruction is analyzed to extract:
 - Opcode (6 bits)
 - Source Register 1 (rs - 5 bits)
 - Source Register 2 (rt - 5 bits)

- Destination Register (rd - 5 bits)
- Shift Amount (shamt - 5 bits)
- Function Code (funct - 6 bits)
- Immediate Value (16 bits - sign-extended if necessary)

3. Execute:

- **ALU Operations:** The ALU performs the operation specified by the ALUOp control signal. Operands are fetched from the register file or sign-extended immediate value. The Zero flag is set if the result is zero.
- **Memory Access:** For load and store instructions, the memory address is calculated by adding the base address (from rs) and the sign-extended offset. Memory read or write operations are performed accordingly.
- **Branching:** The branch target address is calculated. If the branch condition is met (e.g., Zero flag set for beq), the PC is updated to the target address.

4. Memory Access (if required):

- **lw (Load Word):** Data at the calculated memory address is fetched and written to the destination register.
- **sw (Store Word):** Data from the source register is written to the calculated memory address.

5. Write Back (if required):

- The result of the ALU operation or the data loaded from memory is written back to the destination register, if enabled by RegWrite.

3.6 ALU Implementation: The Computational Engine

The ALU is responsible for performing arithmetic, logic, and comparison operations.

- **Implementation:**
 - Encapsulated within an ALUOperation(operand1, operand2, ALUOp) function.
 - ALUOp, a 3-bit control signal, dictates the specific operation.
- **Operations Supported:**
 - Arithmetic: add, addi, sub, mul
 - Logical: and, or, slt, sll, ori
- **Zero Flag:** Set if the result of the ALU operation is zero.

3.7 Memory Access Implementation: Interacting with Memory

lw and sw instructions handle interactions with the simulated memory.

- **Address Calculation:**
 - $\text{address} = \text{base_register} + \text{sign_extended_offset}$

- **lw (Load Word):**
 - Fetches data from the calculated memory address and writes it to the destination register.
- **sw (Store Word):**
 - Writes data from the source register to the calculated memory address.

3.8 Branching Implementation: Altering Control Flow

Branch instructions introduce decision points in program execution.

- **beq (Branch if Equal):**
 - Compares the values in two source registers using the ALU.
 - If equal (Zero flag set), the PC is updated: $PC = PC + 4 + (\text{sign_extended_offset} \ll 2)$.
 - The $\ll 2$ accounts for word-aligned branch offsets.
- **j (Jump), jal (Jump and Link), jr (Jump Register):**
 - These instructions unconditionally modify the PC based on their specific behavior (direct address, address from a register, or return address in \$ra).

3.9 System Call Implementation: Interfacing with the Host

System calls provide a mechanism for the simulated program to interact with the host operating system, enabling basic input/output and program termination.

- **Mechanism:**
 - When a syscall instruction is encountered:
 - The simulator checks the value in the \$v0 register (system call code).
 - A predefined function corresponding to the system call code is executed.
- **Supported System Calls:**
 - **Print Integer (1):** Prints the value in \$a0 to the console.
 - **Print String (4):** Prints the null-terminated string starting at the address in \$a0.
 - **Read Integer (5):** Reads an integer from the console and stores it in \$v0.
 - **Read String (8):** Reads a string (up to the length specified in \$a1) and stores it at the address in \$a0.
 - **Exit (10):** Terminates the program.

4. Challenges and Solutions

4.1 Two-Pass Assembly and Symbol Table Management: Resolving Addresses

Challenge: Resolving label addresses during code generation, as forward references to labels pose a challenge in a single-pass approach.

Solution: A two-pass assembly process elegantly addresses this:

- **Pass 1:** Focuses on building the symbol table, mapping labels to their addresses.
- **Pass 2:** Utilizes the populated symbol table to resolve label addresses when encoding instructions.
- `std::map` proves efficient for managing the symbol table due to its fast search and insertion operations.

4.2 Handling Pseudo-Instructions: Expanding Convenience

Challenge: MIPS assembly includes pseudo-instructions (e.g., `la`, `li`) for programmer convenience, but these are not directly executable by the hardware.

Solution: Pseudo-instructions are translated into equivalent sequences of actual MIPS instructions.

- **Example:** `la $t0, label` (load address of 'label' into register `$t0`) is typically implemented using a `lui` (load upper immediate) followed by an `ori` (or immediate) instruction.

4.3 Alternative Solutions Considered & Lessons Learned: Reflecting on the Journey

- **Hash Table for Symbol Table:** While `std::map` provided adequate performance, a hash table could potentially offer faster lookups, especially for programs with a large number of labels. However, the complexity overhead of implementing a hash table was deemed outweighed by the performance gains in this context.
- **Macro Expansion for Pseudo-Instructions:** An alternative approach involving a preprocessor to expand pseudo-instructions before the assembly process was considered. This approach offers a cleaner separation of concerns but introduces additional complexity in toolchain management.
- **More Complex Memory Model:** Implementing virtual memory and caching mechanisms would enhance the simulator's realism and allow for more complex program execution. However, these features were deemed outside the scope of the initial implementation, as the focus was on core MIPS concepts.

Lessons Learned:

- **Modular Design:** Essential for managing complexity, promoting code reuse, and simplifying debugging.
- **Thorough Testing:** Crucial for ensuring simulator correctness, especially given the intricate interactions between components.
- **Clear Documentation:** Facilitates understanding, collaboration, and future maintenance.

5. Testing and Results

Testing Results:

1. `testCode1.txt` Expected behavior: Perform ADD, SUB, OR, AND, and SLT operations.
 - Store 10 in `$t0` and 20 in `$t1`
 - ADD result in `$t2` (expected: 30)

- SUB result in \$t3 (expected: -10)
- AND result in \$t4 (expected: 0)
- OR result in \$t5 (expected: 30)
- SLT result in \$t6 (expected: 1)

Actual output: Values match the expected results.

```
Register : 00111 Value : 0
Register : 01000 Value : 10
Register : 01001 Value : 20
Register : 01010 Value : 30
Register : 01011 Value : 10
Register : 01100 Value : 0
Register : 01101 Value : 30
Register : 01110 Value : 1
Register : 01111 Value : 0
Register : 10000 Value : 0
```

2. testCode2.txt Expected behavior: Execute la, lw, addi, beq, sub, and j instructions.

- Store 5 in \$t0 and \$t1
- ADDI result in \$t2 (expected: 15)
- BEQ should branch to equal_case
- "sub \$t3, \$t0, \$t1" should not execute due to branching
- \$t3 should contain 10 from "add \$t3, \$t0, \$t1"

Actual output: Values match the expected results.

```
Register : 01000 Value : 5
Register : 01001 Value : 5
Register : 01010 Value : 15
Register : 01011 Value : 10
Register : 01100 Value : 0
Register : 01101 Value : 0
Register : 01110 Value : 0
```

3. testCode3.txt Expected behavior: Calculate the Nth Catalan number. Result: The code is functioning correctly.

```
Please enter an integer n : 5
42

Please enter an integer n : 10
16796

Please enter an integer n : 8
1430
```

4. testCode4.txt Expected behavior: Calculate the factorial of a given number. Result: The code is functioning correctly.

```
Enter a positive number: 5
Factorial is: 120

Enter a positive number: 8
Factorial is: 40320

Enter a positive number: 10
Factorial is: 3628800
```

5. testCode5.txt Expected behavior: Implement a calculator supporting addition, subtraction, and multiplication based on user input. Result: The code is functioning correctly.

```
Enter first number: 125
Enter second number: 25

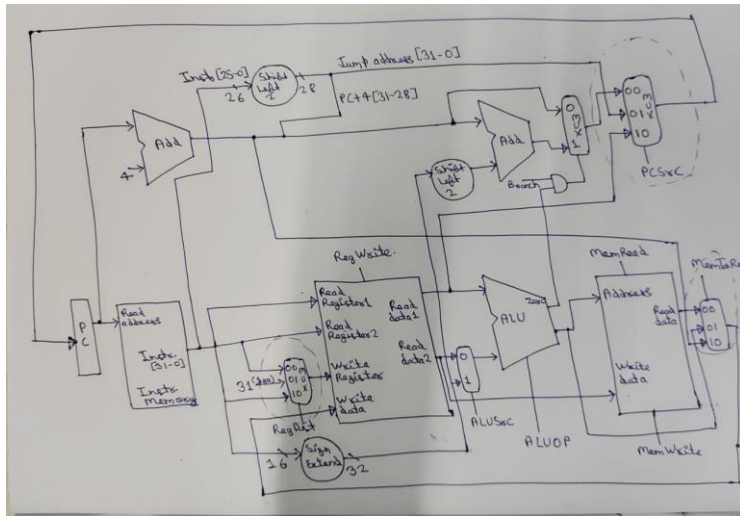
Select operation:
1. Add
2. Subtract
3. Multiply
Enter choice (1-3): 1

Result: 150
```

All test cases have passed and are working as expected.

6 Assumptions

- We have implemented all these instructions to be simulated {"syscall", "add", "sub", "mul", "and", "or", "ori", "sll", "slt", "lw", "beq", "move", "addi", "j", "jal", "jr", "li", "la", "lui", "sw"}
- But here, note that "lw" does not handle when it is provided with a label, i.e., lw \$t1, and LABEL will not work. But the same result can be achieved by la &at, LABEL and then executing lw \$t1, 0(\$at). Both yield the same result.
Now, since we are implementing extra instructions, we had to change the Processor design a little (as compared to class one). Below is the picture of the processor and control signals corresponding to each instruction. And here is the reference we used for this (https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/964e8d2c1085754ba5ed2eba48269a0b_l05_singlecycle.pdf)



RegDst	PCSrc	MemToReg
00 - Instruction[20-26]	00 - Reg, PC+4	00 - Read data
01 - 31 (\$ra)	01 - j and jal	01 - ALU
10 - Instruction[15-16]	10 - jx	10 - PC+4

ALUOp
000 - add
001 - sub
010 - mul
011 - or
100 - left shift
101 - Set
110 - and

- Below is the picture showing the structure of MIPS program our Simulator handle

```
.data
// This section only support .asciiz, .word and .space

.text
// Please Don't include .globl only include given Instruction
// This section support only the Instruction listed
In the assumption. In syscall it currently only support
read and write of string and int and termination of program
```

7. Conclusion and Reflections

This project successfully developed a functional MIPS simulator capable of assembling and executing a subset of the MIPS I instruction set. The simulator accurately emulates core architectural features, including arithmetic/logic operations, memory access, control flow instructions, and system calls. The comprehensive testing methodology employed validates the correctness and reliability of the implementation.

8. Team Members and Contributions:

- Aniket:** Served as the project lead and main developer, managing the overall implementation.
- Rohan:** Developed the parsing module of the simulator, conducted the analysis, and authored the project report.
- Yashraj:** performed extensive testing, successfully identifying and rectifying both bugs and logical errors.